SIMATS ENGINEERING ASSESSMENT TOOL 2: Scenario-Based

COURSE NAME: Cloud Computing and Big Data Analytics

COURSE CODE: CSA15

COURSE OUTCOME COVERED: CO2: Analyze virtualization architectures to identify performance and security issues in cloud computing environments. (BL3)

Assessment Tool Weightage: 20%

Dave's Taxonomy: Manipulation (Level 2)

Question Count: 5

Total Marks: 25

Code of Conduct

I __Manoj M (192511060)____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: __Manoj M___

Assessment Tasks

Scenario-Based Questions

1. A media company wants to migrate its video streaming platform to the cloud. The system must handle millions of concurrent users with minimal buffering.

- Design a cloud architecture using scalability and caching techniques.
- Explain how load balancing and auto-scaling can ensure smooth streaming.
- Discuss cost optimization strategies for such high-demand workloads.

2. A fintech startup is deploying applications using containers across multiple cloud providers. They face challenges in maintaining consistency and managing deployments.

- Propose a solution using container orchestration and multi-cloud strategies.
- Explain how service discovery and scaling can be handled efficiently.
- Highlight the benefits and risks of a multi-cloud deployment model.

3. An e-learning platform experiences data loss due to accidental deletion in the cloud. The organization needs a robust backup and disaster recovery strategy.

- Design a solution using cloud-native backup, replication, and versioning.
- Explain how recovery time objective (RTO) and recovery point objective (RPO) are achieved.

- Provide steps to ensure business continuity during failures.

4. A government agency is adopting a hybrid cloud model for sensitive and non-sensitive data. Some applications must remain on-premises due to compliance requirements.

- Design a hybrid architecture that ensures secure communication between environments.
- Explain how identity management and encryption should be implemented.
- Discuss challenges in managing hybrid cloud environments.

5. A SaaS provider needs to ensure high availability and fault tolerance for its application. The system must continue operating even if one region goes down.

- Propose a multi-region deployment strategy in the cloud.
- Explain how data replication and failover mechanisms should be configured.
- Discuss monitoring and alerting strategies for proactive issue detection.

Scenario-Based Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Scenario	20%	Demonstrates complete understanding of the scenario context	Good understanding with minor gaps	Partial understanding	Fails to understand the scenario
Identification of Risks / Issues	20%	Identifies all major issues correctly	Identifies most issues	Limited issue identification	Misses major issues
Application of Concepts	25%	Applies virtualization concepts effectively	Applies concepts with minor mistakes	Partially correct application	Weak application
Decision Justification	20%	Decisions are well justified and technically strong	Reasonable justification	Weak justification	No useful justification
Clarity	15%	Clear and well-organized response	Generally clear	Somewhat unclear	Poorly presented

ANSWERS

Question 1: Cloud Video Streaming Platform

Understanding of the Scenario

A media company is moving its video streaming platform to the cloud. The platform must serve millions of concurrent viewers across different geographies with minimal buffering. Demand is highly uneven: new releases, live events and weekends create sudden traffic spikes, so the design must scale quickly and deliver content close to the viewer.

Risks and Issues Identified

- Sudden traffic spikes can overload the origin servers and cause buffering.
- High latency for users who are far from the data centre.
- Very high bandwidth (data egress) cost for video delivery.
- Large, fast-growing storage needs for multiple video resolutions.
- Single points of failure and piracy / unauthorised access to content.

(a) Cloud Architecture Using Scalability and Caching

The proposed architecture is layered so that every layer can scale on its own:

1. **Global DNS and CDN edge layer:** Geo-based DNS routes each viewer to the nearest Content Delivery Network (CDN) edge location, which caches popular video segments.
2. **Load balancer layer:** A Layer-7 application load balancer spreads requests across several Availability Zones.
3. **Application layer:** Stateless microservices (login, catalogue, playback, recommendation) run in auto-scaling virtual machine groups or containers.
4. **Caching layer:** An in-memory cache (Redis / Memcached) stores sessions, metadata and watch lists to reduce database load.
5. **Data layer:** Master videos are stored in object storage (S3 / Blob). User and billing data sits in a managed relational database with read replicas, and watch history in a NoSQL database.
6. **Media processing pipeline:** Uploaded videos are placed in a queue and transcoded by worker nodes into multiple bit-rates (HLS / MPEG-DASH) for adaptive bit-rate streaming.

Caching techniques used: CDN edge caching, an origin shield to protect the origin, inmemory caching for metadata, browser caching with suitable TTL values, and pre-warming of the cache before big releases.

Virtualization concepts applied: Workloads run on virtual machines or containers created on a hypervisor. This gives resource isolation between tenants and allows new instances to be started from the same image within minutes.

(b) Load Balancing and Auto-Scaling for Smooth Streaming

- **Load balancing:** Incoming requests are distributed evenly using round-robin or leastconnections. Health checks automatically remove unhealthy servers, and latency-based routing sends users to the closest region.
- **Auto-scaling:** Instances are added or removed according to metrics such as CPU use, requests per target and number of active streams (target-tracking policy).
- **Scheduled and predictive scaling:** Extra capacity is added before known peaks such as a movie premiere or a live match.
- **Adaptive bit-rate streaming:** The player lowers or raises video quality according to the viewer's network speed, which prevents buffering.

Together, the load balancer removes uneven load, auto-scaling supplies capacity before servers saturate, and the CDN keeps most traffic away from the origin. This gives continuous playback even at peak load.

(c) Cost Optimisation Strategies

Strategy	How it reduces cost
Reserved / Savings Plans	Cheaper pricing for the steady baseline load that runs all year.
Spot instances	Up to 70–90% cheaper for fault-tolerant transcoding jobs.
On-demand + auto-scaling	Pay for extra capacity only during peaks and release it afterwards.
Storage tiering	Move old or rarely watched videos to infrequent-access or archive storage by lifecycle rules.
High CDN hit ratio	Reduces origin traffic and data-egress charges; committed-use CDN contracts give discounts.
Efficient codecs (H.265 / AV1)	Same quality at lower bit-rate, so less bandwidth and storage.
Right-sizing and budgets	Remove idle resources, tag workloads and set budget alerts.

Justification of Decisions

A CDN is chosen because video is read-heavy and cacheable, so it solves latency and cost together. Stateless services behind a load balancer are chosen because they can scale horizontally without any session dependency. A mix of reserved, on-demand and spot capacity balances cost against reliability.

Architecture Flow (Request Path)

Step	Component	Role in Smooth Streaming
1	Viewer device / player	Requests the manifest file and selects the video quality using adaptive bit-rate.
2	Geo DNS	Resolves the request to the nearest edge location.
3	CDN edge cache	Serves cached video segments directly (cache hit); forwards a miss to the origin shield.
4	Origin shield and load balancer	Merges repeated misses into one request and balances load across zones.
5	Application servers	Authenticate the user, check subscription and return the playback URL.
6	Object storage and database	Store master video files, metadata and user data.

Performance and Security Issues in the Virtualized Environment

- **Performance:** Noisy-neighbour effect on shared hosts, virtualization overhead and network I/O limits can affect transcoding speed. These are handled by choosing compute- or network-optimised instance types, placement groups and enhanced networking.
- **Cold start delay:** New instances take time to boot. Pre-built images, warm pools and containers reduce start-up time during sudden spikes.
- **Security:** Content is protected with Digital Rights Management (DRM), signed URLs and tokens, HTTPS everywhere, a Web Application Firewall and DDoS protection (for example AWS Shield). Tenant isolation is maintained by the hypervisor and private network segments.
- **Availability:** Deploying across multiple Availability Zones removes a single point of failure; health checks replace failed servers automatically.

Monitoring and Performance Measurement

Quality of experience is tracked using metrics such as start-up time, rebuffering ratio, bit-rate switches, CDN cache-hit ratio and error rate. Dashboards and alarms trigger scaling actions or traffic redirection before viewers notice a problem.

Conclusion

A CDN-first, stateless and auto-scaling architecture lets the platform serve millions of users with very little buffering. Load balancing and auto-scaling manage capacity, caching reduces latency and origin load, and a mix of pricing models with storage tiering keeps the cost of highdemand workloads under control.

Question 2: Containers Across Multiple Cloud Providers

Understanding of the Scenario

A fintech start-up runs containerised applications on more than one cloud provider. Each provider has different tools and APIs, which leads to inconsistent environments and difficult deployments. Because the business handles financial data, security and compliance are also critical.

Risks and Issues Identified

- Configuration drift and "works on one cloud but not on another" problems.
- Manual, error-prone deployments with no single control point.
- Different security, networking and identity models in each cloud.
- Fragmented monitoring, and vendor lock-in if provider-specific services are used.
- Compliance requirements such as PCI-DSS and data residency.

(a) Solution Using Container Orchestration and Multi-Cloud Strategy

1. **Standard container images:** Applications are built once as Docker images, signed and stored in a central registry, so the same image runs everywhere.
2. **Kubernetes as the common layer:** Managed Kubernetes (EKS, AKS, GKE) is used on every cloud, so deployment objects and APIs stay identical.
3. **Infrastructure as Code:** Terraform creates clusters, networks and policies in the same way on every provider.
4. **GitOps and CI/CD:** Argo CD or Flux deploys from a single Git repository (Helm charts), which removes drift and makes rollbacks easy.
5. **Multi-cluster management and policy:** Tools such as Rancher or Anthos give one control plane, while OPA Gatekeeper enforces common security policies.

6. **Secrets and security:** HashiCorp Vault for secrets, image scanning in the pipeline, and a service mesh (Istio) for mutual TLS between services.

Multi-cloud strategy: Run the main workload on one provider and keep a second provider as active-active or standby. A global traffic manager routes users and handles failover. Cloudagnostic and open-source components are preferred to avoid lock-in.

Virtualization concepts applied: Containers share the host kernel and are lighter than VMs, so they start in seconds and are portable. Namespaces and cgroups provide isolation and resource limits.

(b) Service Discovery and Scaling

- **Service discovery:** Kubernetes Services and CoreDNS give every service a stable DNS name, even though pod IP addresses keep changing. Ingress controllers expose services externally.
- **Cross-cloud discovery:** A multi-cluster service mesh or Consul with mesh gateways lets services in one cloud find and call services in another securely.
- **Pod scaling:** The Horizontal Pod Autoscaler adds pods based on CPU or custom metrics; KEDA scales on queue length or transaction rate.
- **Node scaling:** Cluster Autoscaler or Karpenter adds and removes worker nodes as pods need capacity.
- **Safe releases:** Readiness and liveness probes, rolling updates and canary releases avoid downtime.

(c) Benefits and Risks of Multi-Cloud

Benefits	Risks
Avoids vendor lock-in and improves bargaining power	Higher operational complexity
Higher resilience if one provider fails	Need for skilled staff on several platforms
Choose best service from each provider	Higher cost (data transfer between clouds, duplicated tools)
Meets data-residency and compliance needs	Inconsistent security models and larger attack surface
Better performance by placing workloads near users	Latency and data-consistency issues between clouds

Mitigation: Use IaC, a single identity provider, centralised logging and monitoring (Prometheus, Grafana), and standard policies.

Justification of Decisions

Kubernetes with GitOps is selected because it gives one consistent, declarative way to deploy on any cloud. Terraform and policy-as-code make the environments repeatable and auditable, which suits a regulated fintech business. The added complexity of multi-cloud is accepted only for the resilience and compliance gains.

Deployment Workflow (CI/CD with GitOps)

Step	Activity	Outcome
1	Developer commits code to Git	Single source of truth for application and configuration.
2	CI pipeline builds and tests	Docker image is built, unit-tested and scanned for vulnerabilities.
3	Image pushed to registry	Signed, versioned image available to all clouds.
4	GitOps controller detects change	Argo CD / Flux compares the desired state with the cluster state.
5	Rolling or canary deployment	New version is released gradually on each cluster.
6	Monitoring and rollback	Automatic rollback if error rate or latency increases.

Performance and Security Issues

- **Container isolation:** Containers share the host kernel, so a kernel vulnerability can affect all of them. Mitigations include non-root containers, read-only file systems, seccomp / AppArmor profiles, minimal base images and, for highly sensitive workloads, sandboxed runtimes or running containers inside VMs.
- **Network security:** Kubernetes Network Policies, a service mesh with mutual TLS, and encrypted links between the cloud providers protect traffic.
- **Supply-chain security:** Image scanning, signed images and a private registry stop untrusted images from running.
- **Access control:** Kubernetes RBAC and a central identity provider with multi-factor authentication; secrets are never stored inside images.

- **Performance:** Cross-cloud traffic adds latency and egress cost, so chatty services are placed in the same cluster and resource requests and limits are set for every pod.

Compliance for a Fintech Environment

Payment data must follow PCI-DSS and local financial regulation. Audit logs, encryption of data at rest and in transit, data-residency controls (keeping customer data in an approved region) and regular penetration tests are built into the pipeline so that compliance is continuous rather than a one-time activity.

Conclusion

Standardised containers, Kubernetes, Infrastructure as Code and GitOps give the start-up consistency and controlled deployments across providers. Service discovery and autoscaling keep the system efficient, while careful security and cost management reduce the main risks of the multi-cloud model.

Question 3: Backup and Disaster Recovery for an E-Learning Platform

Understanding of the Scenario

An e-learning platform lost data because someone accidentally deleted it in the cloud. The platform holds course content, student records and assessment results, so the organisation needs backups that cannot be lost by a single mistake, together with a tested disaster-recovery plan.

Risks and Issues Identified

- No versioning, so deleted or overwritten files cannot be recovered.
- Human error and malicious or ransomware deletion.
- Backups kept in the same account or region as the live data.
- No defined RTO / RPO and no tested recovery procedure.
- Excessive permissions that allow deletion of data and backups.

(a) Solution Using Cloud-Native Backup, Replication and Versioning

- **Versioning:** Enable object versioning on storage buckets, with MFA-delete or Object Lock (WORM) so that older versions cannot be removed.
- **Automated backups:** Use a centralised service (AWS Backup / Azure Backup) with policies for daily, weekly and monthly backups. Database snapshots and point-in-time recovery cover the databases.

- **Replication:** Cross-region replication of storage and a cross-region database replica keep a second copy away from the primary region.
- **Isolated backup account:** Backups are stored in a separate account with restricted, least-privilege access and soft-delete protection.
- **Lifecycle and encryption:** Older backups move to archive storage; all backups are encrypted with managed keys.
- **Infrastructure as Code:** The whole environment can be recreated quickly in the recovery region.

This follows the **3-2-1 rule**: three copies of the data, on two types of storage, with one copy off-site.

Virtualization concepts applied: Virtual machine and volume snapshots capture the full state of a server at a point in time, so a VM can be restored or recreated on new hardware.

(b) Achieving RTO and RPO

RPO (Recovery Point Objective) is the maximum amount of data loss that is acceptable, measured in time. It is controlled by backup and replication frequency. **RTO (Recovery Time Objective)** is the maximum time allowed to restore service. It is controlled by the type of recovery set-up and by automation.

DR Pattern	Typical RPO	Typical RTO	Cost
Backup and restore	Hours	Hours	Low
Pilot light	Minutes	Tens of minutes	Medium
Warm standby	Seconds–minutes	Minutes	Medium–High
Multi-site activeactive	Near zero	Near zero	High

For this platform, **warm standby** is recommended, with a target of RPO of 15 minutes or less and RTO of 1 hour or less. RPO is met by continuous database replication and frequent snapshots; RTO is met by pre-built standby servers, automated runbooks, low-TTL DNS failover and regular DR drills.

(c) Steps to Ensure Business Continuity During Failures

1. Detect the failure through monitoring alarms and alert the response team.
2. Declare an incident and activate the disaster-recovery plan.
3. Contain the problem, for example stop further deletions and restrict access.
4. Assess which data and services are affected.

5. Restore from the latest object version or snapshot, or fail over to the standby region.
6. Redirect users through DNS or the load balancer.
7. Validate data integrity and test key functions such as login, courses and exams.
8. Communicate status to students, faculty and management.
9. Fail back to the primary environment once it is stable.
10. Hold a post-incident review and update the runbooks.

Justification of Decisions

Versioning and immutable backups directly solve accidental deletion, while cross-region copies protect against regional failure. Warm standby is chosen because it meets an RTO of about one hour at a reasonable cost, which suits an education platform where exam periods are critical but a full active-active set-up is not essential.

Backup and Recovery Architecture

Layer	Protection Mechanism	Purpose
Object storage (course content)	Versioning, Object Lock, cross-region replication	Recover deleted or overwritten files; protect against ransomware.
Relational database (records, grades)	Automated snapshots, point-intime recovery, cross-region replica	Restore data to any minute within the retention period.
Virtual machines and volumes	Scheduled snapshots stored in another region	Rebuild servers quickly after failure.
Configuration and code	Git repository and Infrastructure as Code	Recreate the environment in the recovery region.
Backup vault	Separate account, encryption, soft-delete, retention lock	Prevent accidental or malicious deletion of backups.

Preventing Accidental Deletion

- Apply least-privilege IAM roles so that only a few administrators can delete production data.
- Require multi-factor authentication and an approval workflow for delete operations.
- Use a recycle-bin or soft-delete feature with a retention period (for example 30 days).
- Enable audit logging (CloudTrail / Activity Log) with alerts on bulk-delete actions.
- Tag resources and use deletion-protection flags on databases and critical buckets.

Testing, Security and Cost

- **Testing:** Restore tests are run every month and a full DR drill every six months. A backup that has never been restored cannot be trusted.
- **Security:** Backups are encrypted, access is logged and restore permissions are separated from delete permissions.
- **Cost:** Lifecycle rules move older backups to cheaper archive storage and delete those that exceed the retention policy.
- **Performance:** Incremental backups and off-peak scheduling reduce the load on the production system.

Conclusion

Versioning, immutable backups and cross-region replication protect the e-learning platform from accidental deletion and regional failure. A warm-standby design with clear RTO and RPO targets, supported by regular testing and a documented continuity plan, ensures students and faculty can continue to learn with minimal interruption.

Question 4: Hybrid Cloud for a Government Agency

Understanding of the Scenario

A government agency holds both sensitive data (such as citizen records) and non-sensitive data. Compliance rules require some applications to stay on-premises, but the agency also wants the flexibility and scalability of public cloud for other workloads. The two environments must work together securely.

Risks and Issues Identified

- Data leakage or non-compliance if sensitive data is placed in the wrong environment.
- Insecure communication between on-premises and cloud.
- Separate identity systems leading to weak access control.
- Inconsistent security policies, tools and monitoring across environments.
- Latency, integration difficulty and shortage of skills.

(a) Hybrid Architecture with Secure Communication

1. **On-premises private cloud:** Hosts sensitive and regulated applications and databases inside the agency data centre.
2. **Public cloud:** Hosts public portals, analytics on anonymised data, and development and testing environments.

3. **Secure connectivity:** A dedicated private link (AWS Direct Connect / Azure ExpressRoute) with an IPsec site-to-site VPN as backup. Traffic does not travel over the open internet.
4. **Network segmentation:** Separate VPC / VNet subnets, DMZ, firewalls, security groups, WAF and IDS / IPS.
5. **API gateway with mutual TLS:** All service-to-service calls between environments pass through a controlled gateway.
6. **Data classification policy:** Every data set is labelled, and the label decides whether it may leave the premises.
7. **Unified management and monitoring:** A single management plane (Azure Arc / Anthos) and a central SIEM collect logs from both sides.

Virtualization concepts applied: The same hypervisor-based virtualization and container platform is used on-premises and in the cloud, so workloads are portable and isolated by tenant and by network.

(b) Identity Management and Encryption

Identity management

- Federate on-premises Active Directory with the cloud identity provider (SAML / OpenID Connect) for single sign-on.
- Make multi-factor authentication mandatory, especially for administrators.
- Apply role-based access control and least privilege; use Privileged Access Management for admin accounts.
- Use conditional access, regular access reviews and complete audit logging.

Encryption

- **Data at rest:** AES-256 encryption using a Key Management Service or Hardware Security Module. Keys for sensitive data stay under the agency's control (BYOK / HYOK).
- **Data in transit:** TLS 1.2 / 1.3 for applications and IPsec for network links.
- Key rotation, certificate management, and tokenisation or masking of personal data.

(c) Challenges in Managing Hybrid Cloud

Challenge	Explanation and Mitigation
Security consistency	Policies differ across sites; use common policy-as-code and Zero Trust.
Compliance and audit	Data must be tracked and proven to stay where required; use classification and central logging.
Integration	Different platforms and APIs; use standard APIs, containers and middleware.
Latency and bandwidth	Slow links affect performance; use dedicated connections and caching.
Data governance	Keeping data synchronised and consistent; define clear ownership and sync rules.
Skills and cost visibility	Staff need both on-premises and cloud skills; use training and cost-management tools.

Justification of Decisions

Keeping sensitive systems on-premises satisfies compliance, while public cloud is used only for non-sensitive workloads to gain scalability. Private links, federated identity with MFA, and customer-controlled encryption keys provide defence in depth, which is appropriate for government data.

D) Placement of Workloads (Data Classification)

Data / Application	Placement	Reason
Citizen identity and tax records	On-premises private cloud	Regulation requires data sovereignty and tight control.
Internal casemanagement systems	On-premises	Compliance and low-latency access to local systems.
Public website and information portal	Public cloud	Needs elasticity and global availability; contains no sensitive data.
Analytics on anonymised data	Public cloud	Needs large temporary computing power.
Development and testing	Public cloud	Quick provisioning at lower cost; uses masked data only.
Backup and disaster recovery copies	Encrypted, in a compliant region	Gives resilience while meeting residency rules.

Security Controls and Zero Trust

- **Zero Trust principle:** Never trust, always verify. Every user, device and service call is authenticated and authorised, even inside the network.
- **Micro-segmentation:** Workloads are separated so that a compromised system cannot reach others easily.
- **Hypervisor and VM security:** Hardened and patched hypervisors, isolated virtual networks and secure VM images reduce the risk of VM escape and side-channel attacks.
- **Monitoring:** A central SIEM and Security Operations Centre analyse logs from both environments and raise alerts on suspicious behaviour.
- **Compliance:** Regular audits, vulnerability scans and penetration tests demonstrate that the agency meets government standards.

Performance and Operational Considerations

Applications that talk to each other frequently are kept in the same environment to avoid latency. Dedicated links provide predictable bandwidth, and caching or data replication of nonsensitive data reduces round trips. Common tools such as Kubernetes, Terraform and Ansible are used on both sides so that operations teams manage one process instead of two.

Conclusion

A hybrid architecture with sensitive workloads on-premises and elastic workloads in the public cloud satisfies both compliance and scalability. Private connectivity, federated identity with multi-factor authentication, strong encryption and unified monitoring secure the communication between the environments, while governance and training address the management challenges.

Question 5: High Availability and Fault Tolerance for a SaaS Provider

Understanding of the Scenario

A SaaS provider must keep its application running even if an entire cloud region fails. This means the application and its data must exist in more than one region, and traffic must move automatically to a healthy region with little or no data loss.

Risks and Issues Identified

- A single region is a single point of failure.
- Data loss or inconsistency because of replication lag during failover. • Slow DNS propagation and manual failover causing long downtime.
- Split-brain, where two regions both accept writes.
- Higher cost and complexity of running multiple regions, and failures that are detected too late.

(a) Multi-Region Deployment Strategy

- **Two or more regions**, each deployed across multiple Availability Zones. The model can be active-active (both serve users) or active-passive (one on standby).
- **Global traffic management:** DNS or a global load balancer (Route 53, Global Accelerator, Azure Front Door) with health checks and latency-based or failover routing.
- **Identical environments:** Infrastructure as Code and CI/CD pipelines deploy the same stateless application stack to every region.
- **Regional load balancers and auto-scaling** in each region; each region is sized so that one region can carry the full load if the other fails (N+1).
- **CDN** in front of the application to absorb traffic and serve static content during failover.

Virtualization concepts applied: Standard VM images and container images allow the same application to be launched in any region quickly, and hypervisor-level isolation separates tenants in the multi-tenant SaaS.

(b) Data Replication and Failover Configuration

Data Type	Replication Approach
Relational database	Cross-region read replica or global database; replica is promoted on failure (asynchronous, RPO of seconds).
Data Type	**Replication Approach**
NoSQL database	Global tables / multi-master with conflict resolution such as lastwriter-wins.
Object storage and files	Cross-region replication with versioning.
Cache and sessions	Rebuilt locally in each region; sessions kept in a replicated store or in tokens.
Messages and queues	Regional queues with replication or re-processing of undelivered messages.

Failover configuration:

1. Define deep health checks (for example a /health endpoint that tests the application and database).
2. Use DNS failover routing with a low TTL (30–60 seconds) so users move quickly.
3. Automate promotion of the standby database and scaling up of standby capacity through runbooks or scripts.
4. Allow only one writer at a time (fencing) to prevent split-brain.
5. Test regularly with game-days and chaos engineering, and plan a controlled failback.

(c) Monitoring and Alerting for Proactive Detection

- **Key metrics:** latency, traffic, error rate and saturation (the four golden signals), plus replication lag, CPU, memory and disk.
- **Centralised logging and distributed tracing** (ELK, CloudWatch, Datadog) to find the root cause quickly.
- **Synthetic monitoring** from several regions to simulate user journeys and detect regional problems early.
- **Alerting:** threshold and anomaly-based alerts with severity levels, sent by SMS, email or paging tools (PagerDuty, SNS) to the on-call team.
- **SLOs and error budgets** to measure reliability against the SLA.

- **Runbooks and post-mortems** so that each incident improves the process.

Justification of Decisions

A multi-region design removes the region as a single point of failure. Asynchronous replication with automated failover gives a very small RPO and a short RTO at an acceptable cost, and proactive monitoring ensures that problems are detected before users are affected.

Active-Active versus Active-Passive

Aspect	Active-Active	Active-Passive
Traffic	Both regions serve users at the same time.	Only the primary serves users; the secondary waits.
RTO	Near zero; no switch-over needed.	Minutes, depending on promotion and DNS.
Cost	Higher; capacity is used in both regions.	Lower; the standby can be kept small (warm standby).
Data consistency	Complex; needs conflict resolution.	Simpler; single writer.
Best suited for	Global users and mission-critical SaaS.	Cost-sensitive systems that tolerate a short outage.

For this SaaS provider, an **active-active design for the stateless application tier** and an **active-passive design for the primary database** is a balanced choice. It gives near-instant failover for the application and simple, safe data consistency.

Failover Sequence When a Region Goes Down

1. Health checks in the global traffic manager fail for the primary region.
2. The monitoring system raises a critical alert and notifies the on-call engineer.
3. Traffic is automatically routed to the healthy region.
4. The standby database replica is promoted to primary.
5. Auto-scaling in the surviving region increases capacity to handle the full load.
6. Operations verify application health and data integrity.
7. When the failed region recovers, data is re-synchronised and traffic is gradually restored (failback).

Performance and Security Considerations

- **Replication lag:** Asynchronous replication is fast but can lose the last few seconds of data. Critical transactions can use synchronous or quorum writes within a region.
- **Latency:** Placing regions near the user base and using a CDN keeps response times low.
- **Security:** Encrypted replication links, region-specific encryption keys, the same IAM and firewall policies in every region, and tenant isolation in the multi-tenant SaaS.
- **Cost control:** Use auto-scaling, reserved capacity for the baseline, and keep the standby region at reduced size until needed.

Conclusion

A multi-region deployment with global traffic management, cross-region data replication and automated failover allows the SaaS application to continue operating when a region fails. Proactive monitoring, clear alerting and regular failover testing ensure that issues are found early and recovery is fast and predictable.